```
foreach ($guitars as $guitar) {
    foreach ($guitar as $prop => $str) {
        print "{$prop}: {$str}\n";
    }
    print "\n";
}
```

```
Output:
make: Ibanez
model: RG-8570z
price: £2100

make: Fender
model: American Ultra Stratocaster
price: £1800
```

The outer `foreach` sets `$guitar` to the first sub-array on iteration one; the inner `foreach` then iterates over each property and value within it. When the inner loop is complete, control is returned to the outer loop for a second iteration.

Of course, for arrays that are nested three-deep, three nested `for` or `foreach` loops could be used. This can quickly lead to unmanageable code, so exercise caution. Deeply nested arrays and/or loops should be regarded with equally deep suspicion! For more complex data structures, a recursive function call is probably more appropriate, which we'll look at later when we explore how to process arrays with our own functions.

Assignment During Iteration

During execution of a `for` loop, we work with an iterator variable (typically `$i`) to access each array element sequentially, and we can do whatever we like with that element (provided it's legal PHP syntax, of course). This includes changing values, for example:

```
$arr = [1, 2, 3];
for ($i = 0, $size = count($arr); $i < $size; $i++) {
    $arr[$i]++;
}
print_r($arr);
```